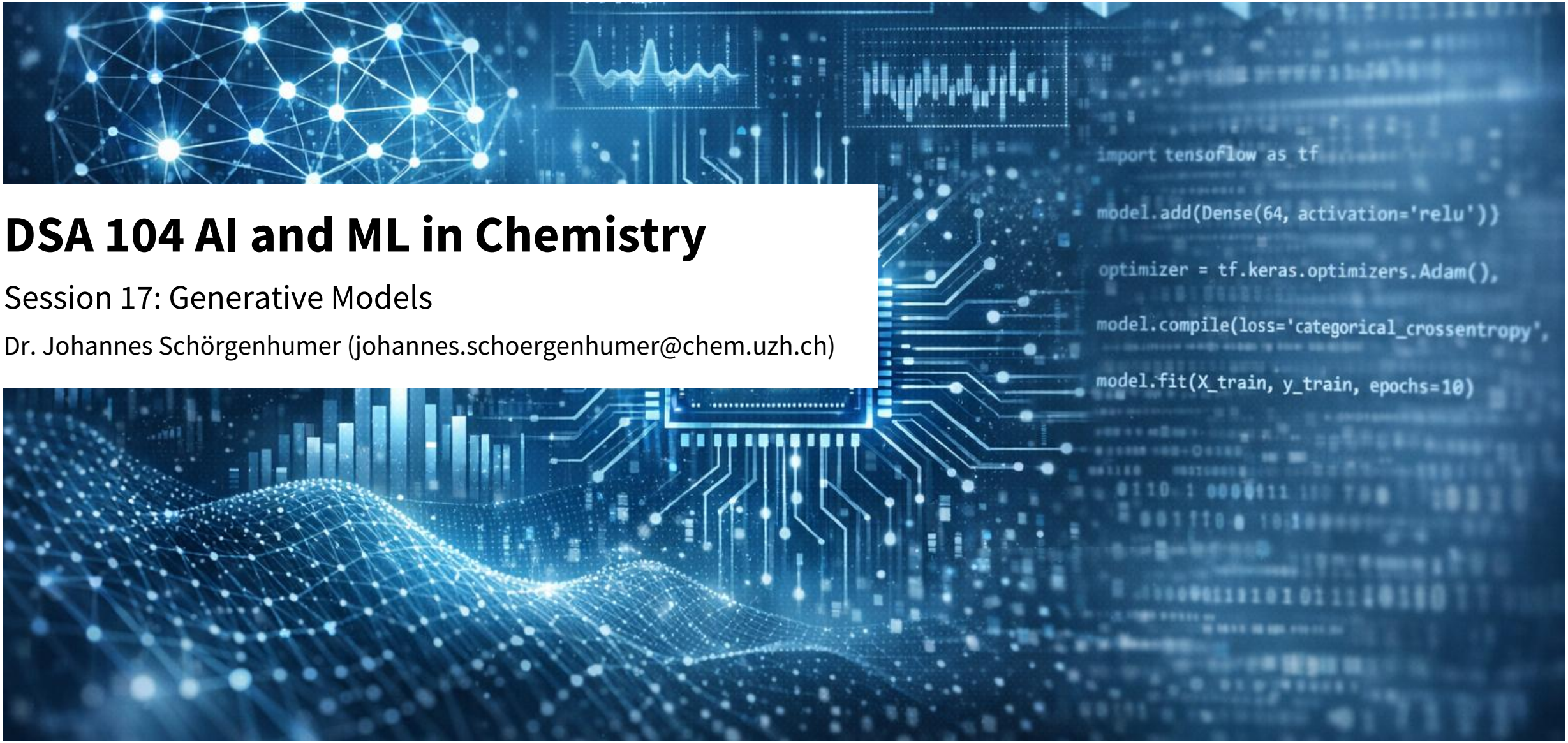
Universität Zürich[UZH]

Department of Chemistry



DSA 104 AI and ML in Chemistry

Session 17: Generative Models

Dr. Johannes Schörgenhumer (johannes.schoergenhumer@chem.uzh.ch)

Points for today

Follow up: Reinforcement learning

Generative models:

— Core concepts

— Layered architecture

— Main families

— Application in life sciences

Reinforcement Learning: Minimalistic example

Have a look at the minimal RL example RL-Bandit.ipynb. This is RL in its purest (and most reduced) form.

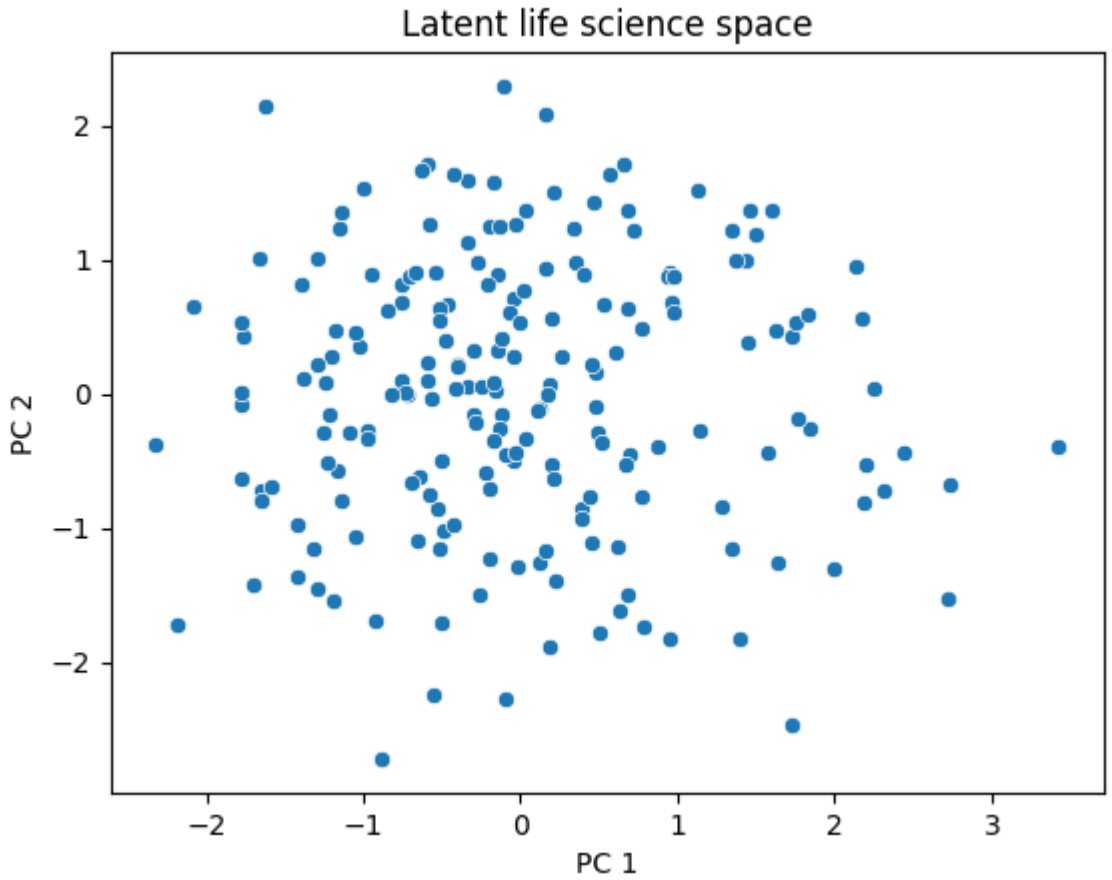
Generative models: Key ideas

- Model learns full data distribution
- Not just predicting one target, but…
- Creating new, realistic examples

Supervised Model	Generative Model
Predicts activity	Generates new molecules
Classifies cell types	Simulates new cells
Needs labels	Labels not strictly necessary

Life science analogies:

- Protein families:
  - many valid sequences
  - not one “correct answer”
- Chemical space:
  - Very, very large, but…
  - structured



Generative models: Core concept

A generative model learns ***p*(data)** and can **sample** from it.

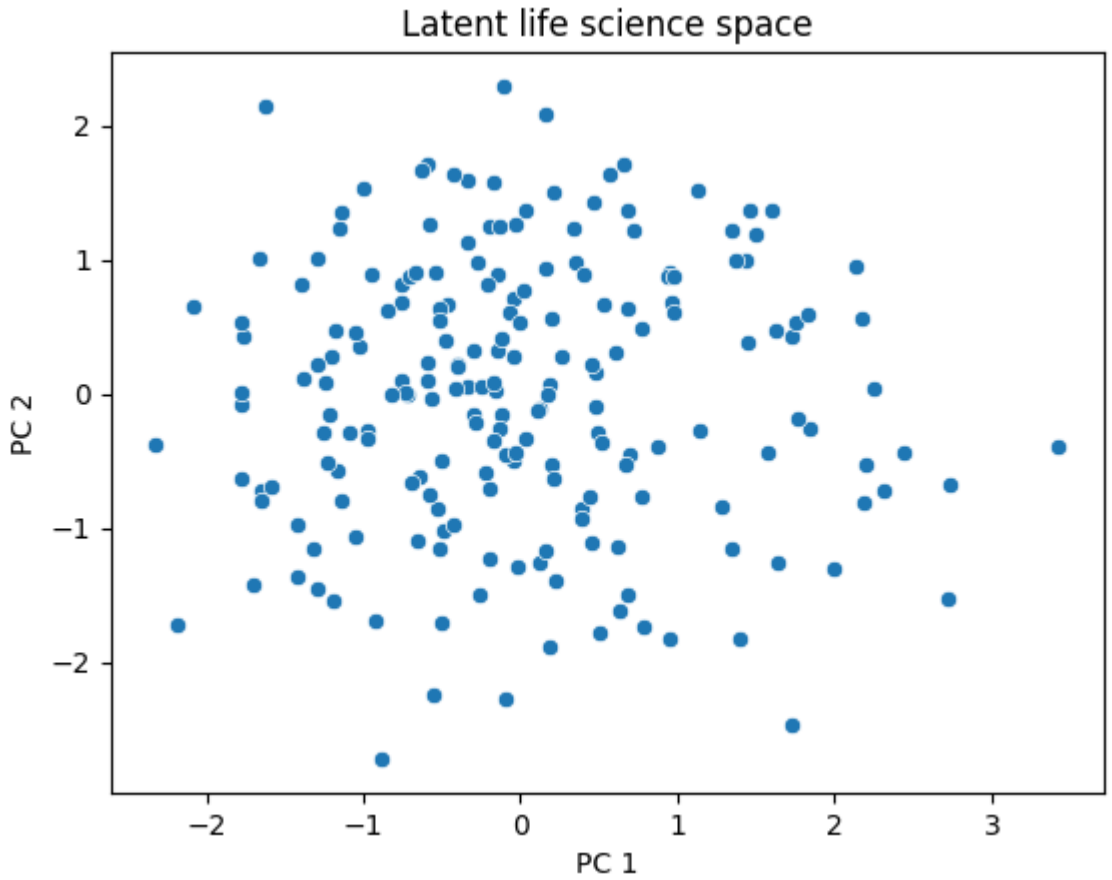
E.g. **latent space**:

— Biology: protein sequences

— Chemistry: molecular structure

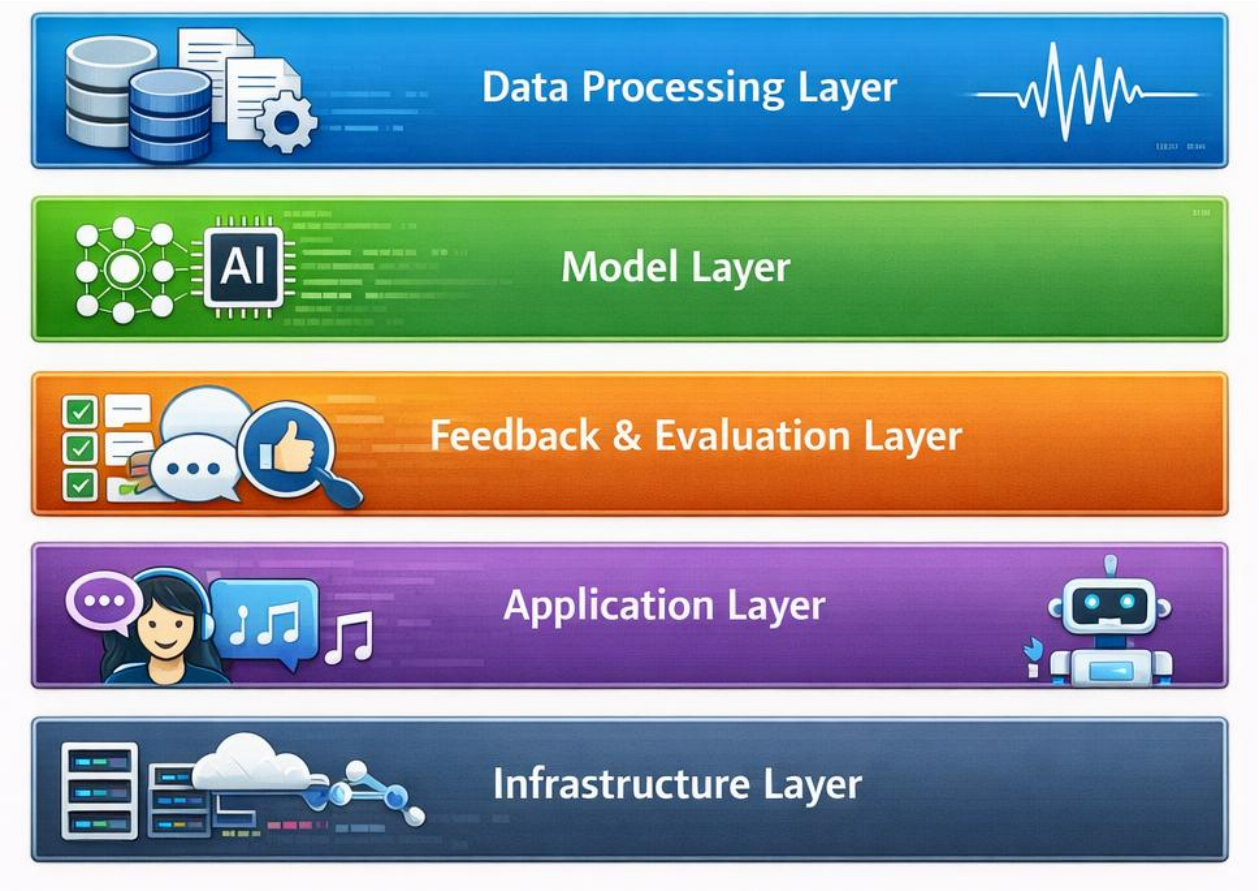
Latent space discussion:

What makes new, generated sample **plausible**?



Generative models: Layered architecture

- **Data processing** layer (Collect, clean, augment, tokenise, …)
- **Model** layer: houses generative models that learn distributions and generate new content, e.g.
  - **Variational Autoencoders (VAEs)**
  - **Generative Adversarial Networks (GANs)**
  - **Diffusion models**
  - Transformers & LLMs
  - Fine-tuned models
- **Feedback and evaluation** layer:
  - Metrics
  - Human-in-the-loop
  - Active learning
- **Application layer** (Chatbots, image synthesisers, …)
- **Infrastructure** Layer (Computational environment)



Adapted from https://www.geeksforgeeks.org/artificial-intelligence/architecture-of-generative-models/
Image generated using Copilot.

Main families of generative models: VAEs

Variational Autoencoders: ***"Compress and generate"***

Core idea:

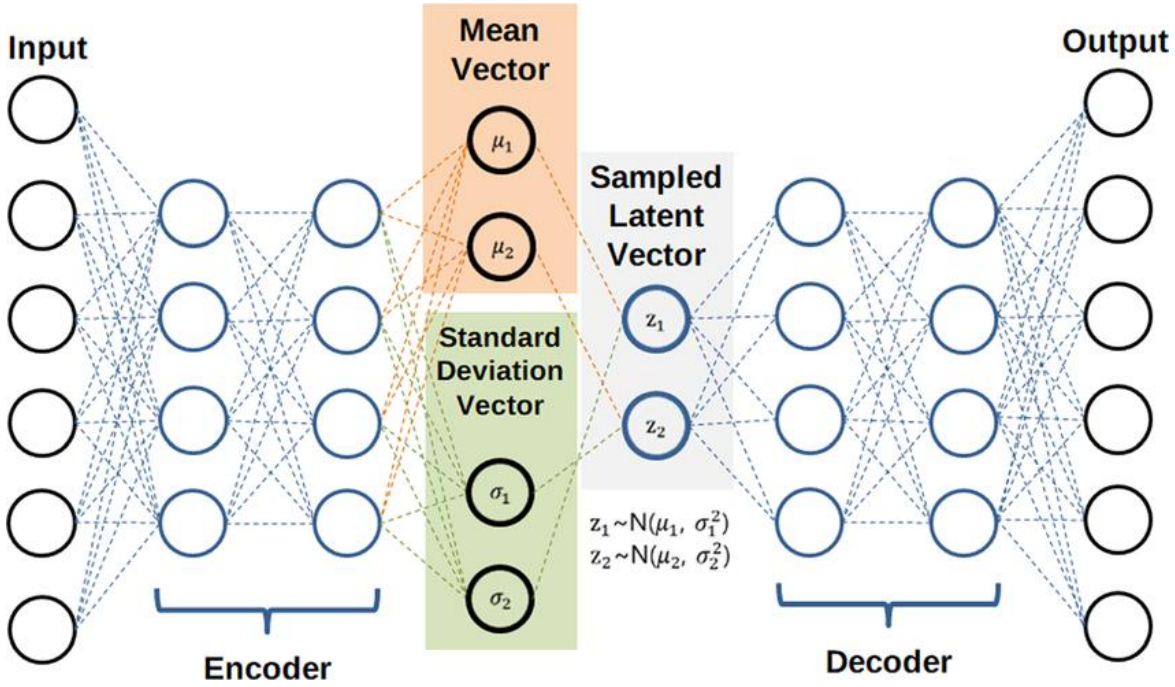
— **Encode** to **latent space**, then **decode**

— Smoothing latent space (Gaussian) enables interpolation

Components = two core networks

— **Encoder:** Maps input data to a parameterised probability distribution in latent space

— **Decoder:** Reconstructs data from sampled points in the latent space



https://www.geeksforgeeks.org/artificial-intelligence/architecture-of-generative-models/
https://www.codecademy.com/article/variational-autoencoder-tutorial-vaes-explained
Picture from: https://www.researchgate.net/publication/364283810_Recent_advances_in_the_application_of_deep_learning_for_fault_diagnosis_of_rotating_machinery_using_vibration_signals

Main families of generative models: VAEs

Training:

— **Reconstruction Loss:** Encourages accurate data reconstruction (e.g. MSE or binary cross-entropy)

— **KL Divergence Loss:** Penalises deviation from the standard Gaussian in latent space

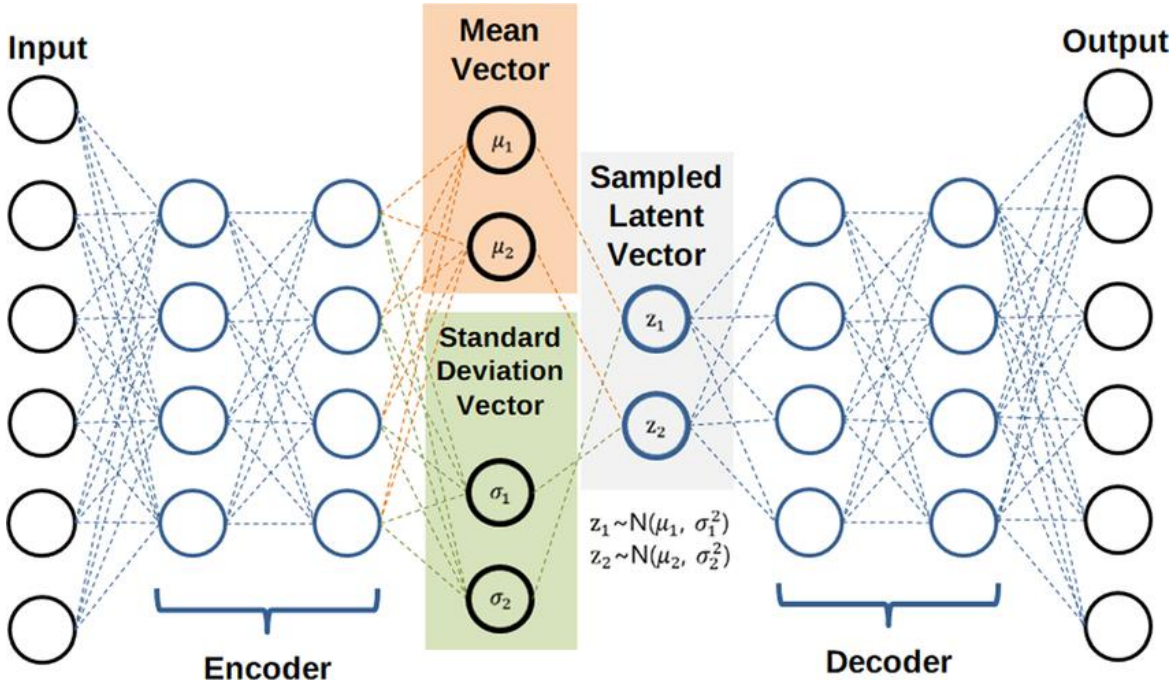
Result:

— Learns **compressed, meaningful latent representations**

— Decoding **random draws** from the latent distribution **generates new plausible samples**

Weaknesses:

— Generated samples may be “blurry” or average



https://www.geeksforgeeks.org/artificial-intelligence/architecture-of-generative-models/
https://www.codecademy.com/article/variational-autoencoder-tutorial-vaes-explained
Picture from: https://www.researchgate.net/publication/364283810_Recent_advances_in_the_application_of_deep_learning_for_fault_diagnosis_of_rotating_machinery_using_vibration_signals

Pytorch VAE demo:

Have a look at the notebook *VAE-demo.ipynb*

The notebook demonstrates by using the MNIST dataset (handwriting) the function of a VAE model, including:

— The visualisation of the latent space

— Visualisation of some generated images

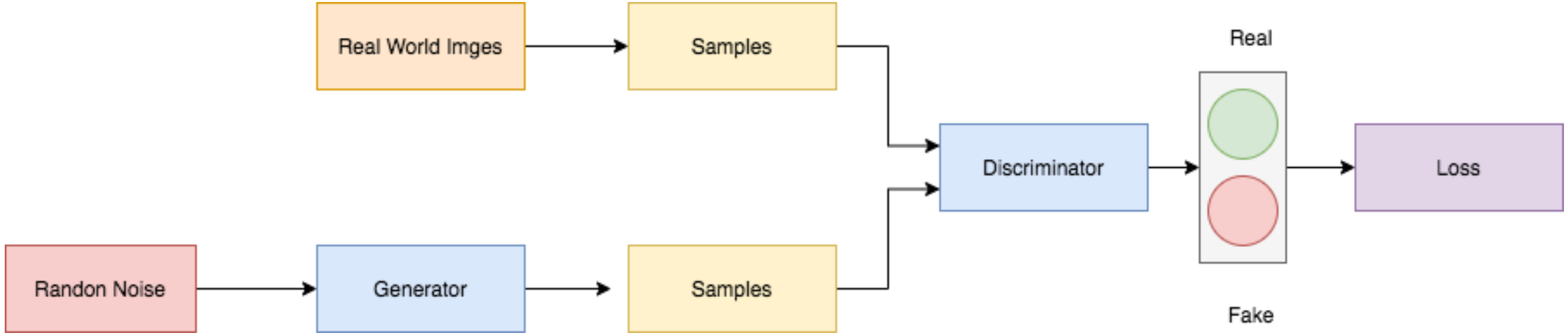
Main families of generative models: GANs

Generative Adversarial Networks: ***"Competition of Generator and Discriminator"***

Components = two core networks:

— **Generator:** Forges new data from noise or variables from latent (e.g. Gaussian) space

— **Discriminator:** Distinguishes real data from generated data



Picture from https://www.geeksforgeeks.org/artificial-intelligence/architecture-of-generative-models/

Main families of generative models: GANs

Adversarial Training, i.e. generator and discriminator trained in opposition:

- Generator tries to fool the discriminator
- Discriminator tries to spot fakes

Loss Functions:

- Binary cross-entropy
- Related adversarial losses

Result: Gradual progress towards sharp, more realistic outputs, as assessed by discriminator performance

Weaknesses:

- Training instability
- **Mode collapse** (creates the same successful data repeatedly)

Adapted from https://www.geeksforgeeks.org/artificial-intelligence/architecture-of-generative-models/

Main families of generative models: Diffusion models

Key idea: **Destroy structure**, learn to undo noise

Key principle:

— **Add noise, then learn to reverse it**

— **Generate by denoising** step-by-step

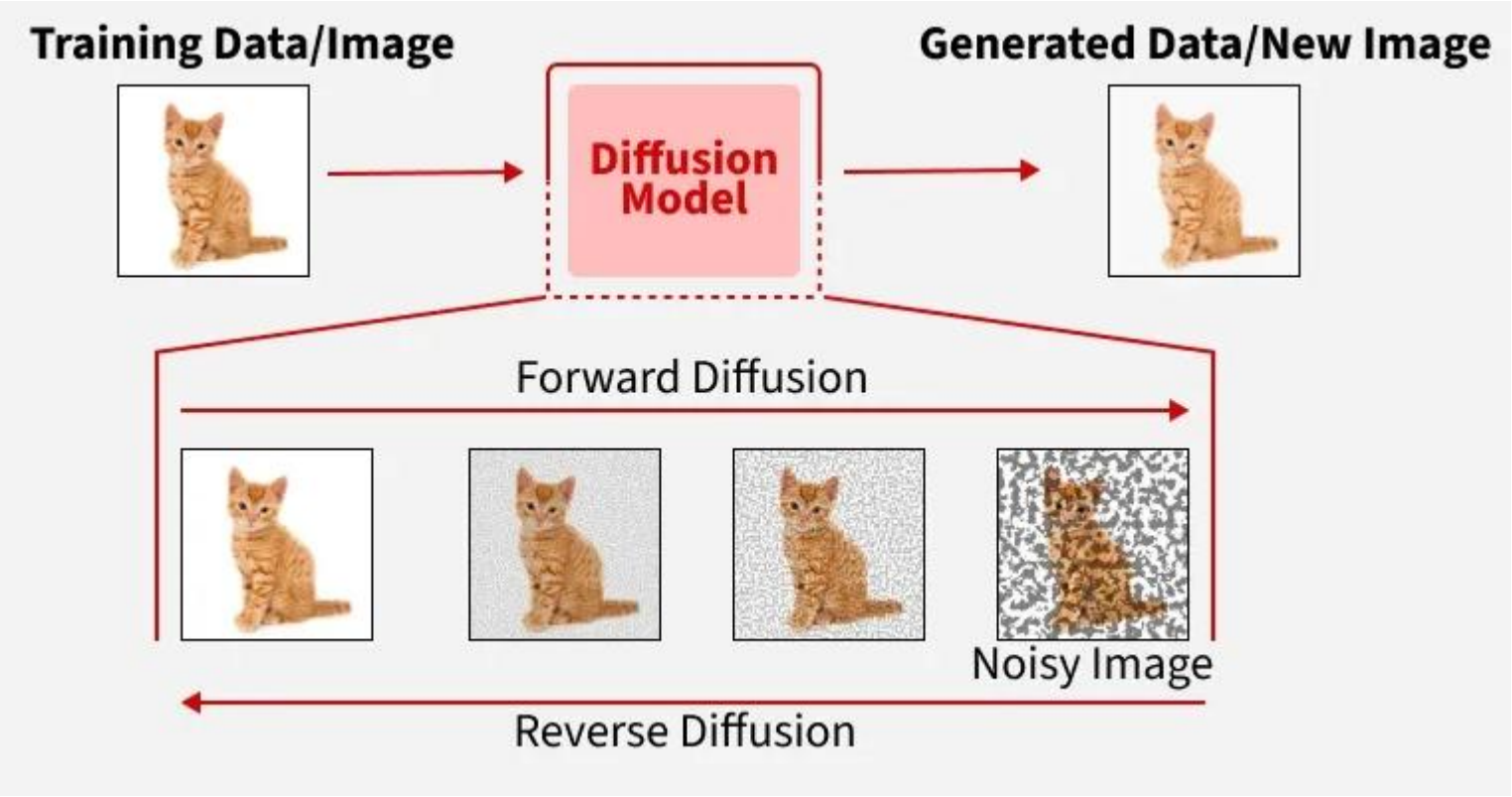
Life science relevance:

— High-quality molecular structures

— Protein structure generation

— Molecular conformations

State-of-the-art image and audio synthesis (e.g., Stable Diffusion, DALL-E 2)



Picture from https://www.geeksforgeeks.org/artificial-intelligence/diffusion-models-in-machine-learning/

Main families of generative models: Language models (Transformers)

Language Models — *"life sciences as a language"*

— DNA, RNA, proteins = sequences

— Molecules, reactions = SMILES

— Same idea as text prediction

Examples:

— Protein language models (e.g., evolution-aware embeddings)

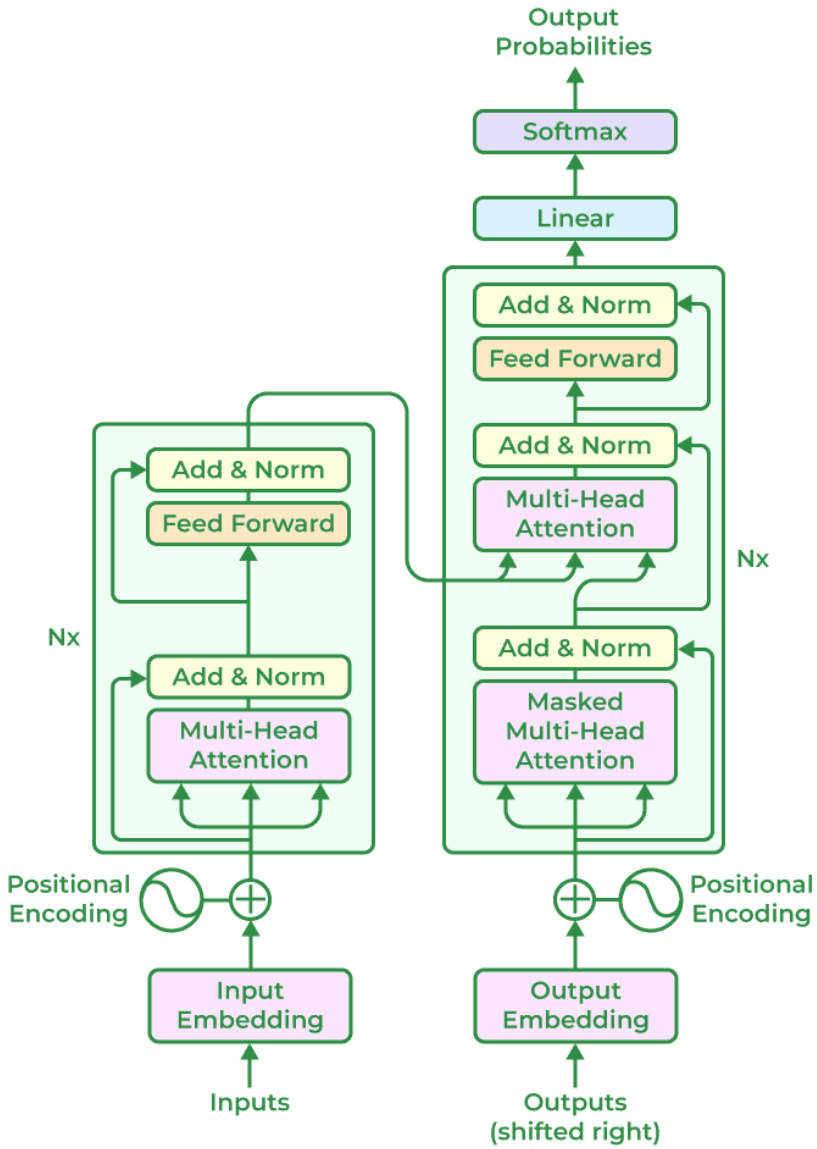
— Designing functional sequences

Additionally, pretrained transformers can also be used for **a lot of other** interesting, **more predictive tasks**, e.g.

— Solubility prediction (May 27!)

— Reaction yield prediction (https://www.nature.com/articles/s42256-023-00788-1)

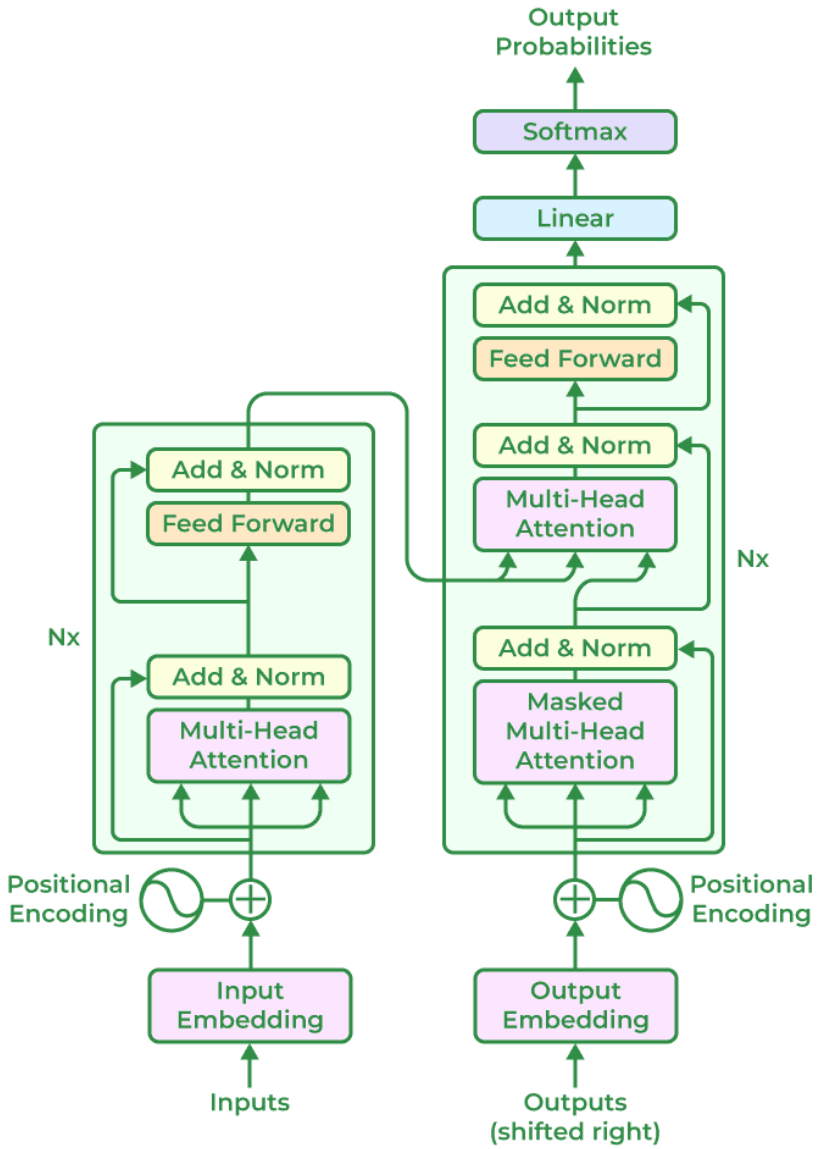
— Chatbot assisted drug design (https://pubs.acs.org/doi/10.1021/acs.jcim.3c01429)



Picture from https://www.geeksforgeeks.org/artificial-intelligence/architecture-of-generative-models/

Main families of generative models: Language models (Transformers)

Tools like ChatGPT, Claude, Gemini, etc. help with a lot of everyday life science tasks:



Picture from https://www.geeksforgeeks.org/artificial-intelligence/architecture-of-generative-models/

LLMs for drug development

Use an LLM of your choice and let it create 5-10 SMILES strings for a new antibiotic molecule or any other specified drug-like candidate

Try prompts like:

— Generate novel molecules similar to X

— Generate a new molecule with a molecular weight between 300 and 400, including at least 4 heteroatoms

Then:

1) Check if the SMILES are actually sound:
   — Using RDKit (notebook provided)
   — Or some online apps, e.g. https://2d-molecular-visualization.streamlit.app/

2) Check if the SMILES strings actually are new molecules (ChEMBL, PubChem, Scifinder, …)

3) Discuss how you could actually determine if those would be potential antibiotics

4) Discuss if these molecules would be feasible (synthesis, biosynthesis, biological source, …)

Generative models: Applications in the life sciences

Discussion up front:

What do you need to pay attention to, when you employ generative models in the life sciences?

Generative models: Applications in the life sciences

What do you need to pay attention to, when you employ generative models in the life sciences?

— Generated does not mean functional!

— Hallucinations could be detrimental

— **Experimental studies necessary**!

— …

Very diverse potential in life sciences:

— Molecular design

— Protein development

— Cell profiles

— …

Generative models: Applications in the life sciences

Molecular Design:

— Drug-like molecules

— Property-conditioned generation

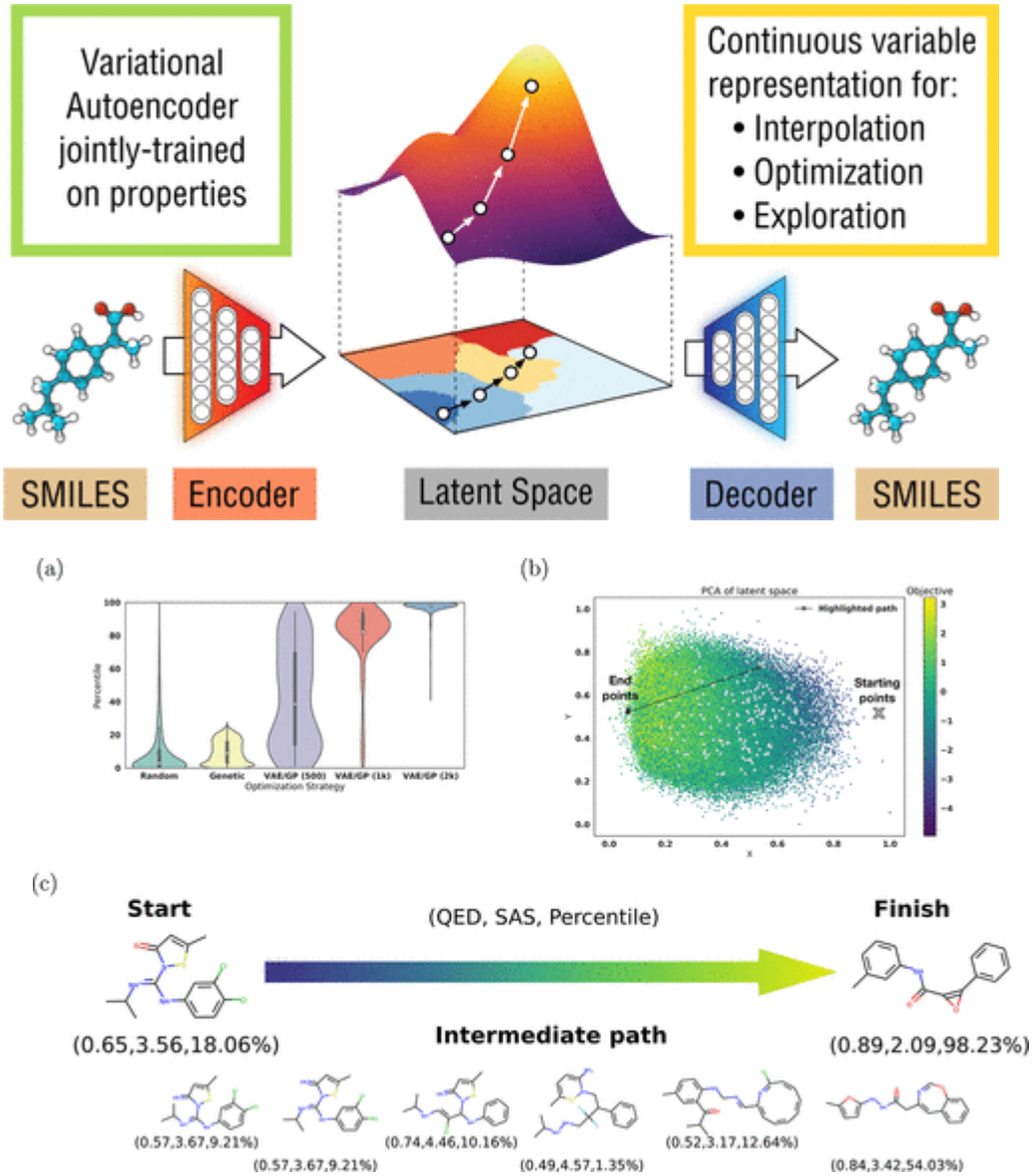
Example:

"*Automatic chemical design using a data-driven continuous representation of molecules*" (Gómez-Bombarelli et al., *ACS Central Science*, **2018**)

— Very early demonstration of a **VAE** for molecule generation

— Introduced idea of a continuous **latent chemical space**

— Jointly trained for **property prediction** (e.g. synthetic accessibility)



https://pubs.acs.org/doi/10.1021/acscentsci.7b00572

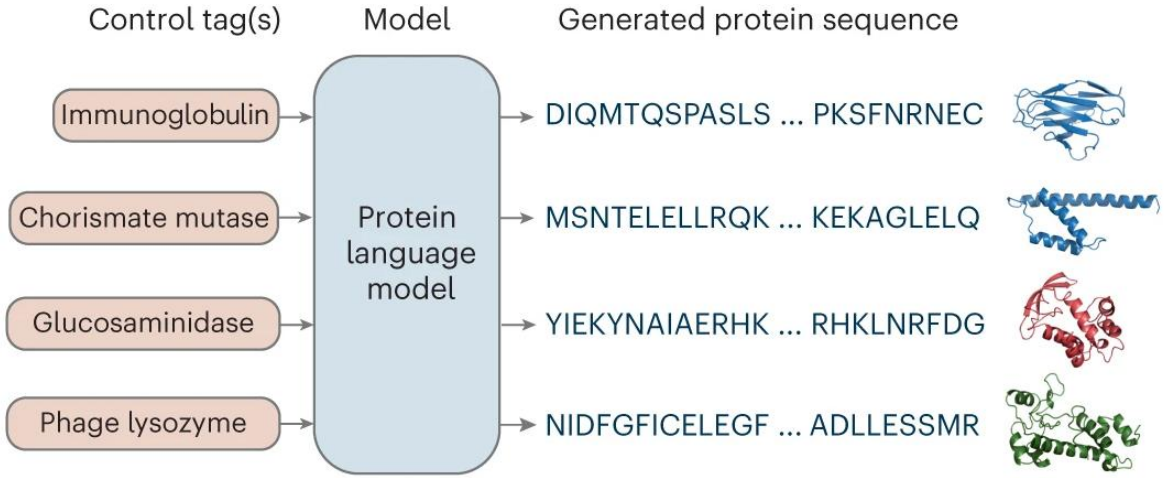
Generative models: Applications in the life sciences

Protein & Sequence Models:

— Protein sequence generation

— Antibody design

— DNA regulatory elements



Example:

"Large language models generate functional protein sequences across diverse families" (Madani et al., *Nature Biotechnology*, **2023**)

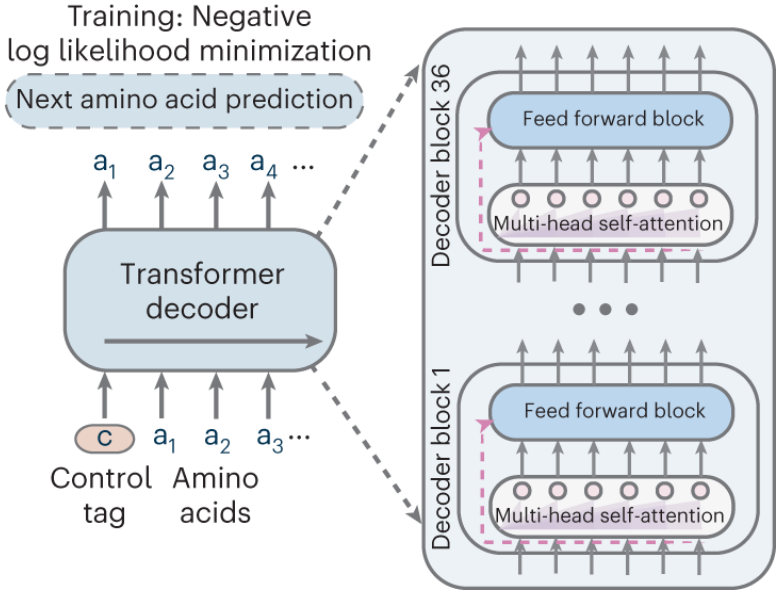
— Landmark demonstration: **language-model generates functional proteins**

— No explicit structural supervision

— Only generative modelling

— Evolutionary **constraints learned from data**

— **Experimental validation** of generated enzymes



https://www.nature.com/articles/s41587-022-01618-2

Generative models: Applications in the life sciences

3D Geometry of molecules:

— Generate conformations

Example:

"GeoDiff: A Geometric Diffusion Model for Molecular Conformation Generation" (Xu et al., *Conference proceedings ICML*, **2022**)

— Single molecular graph = many conformations (difficult problem!)

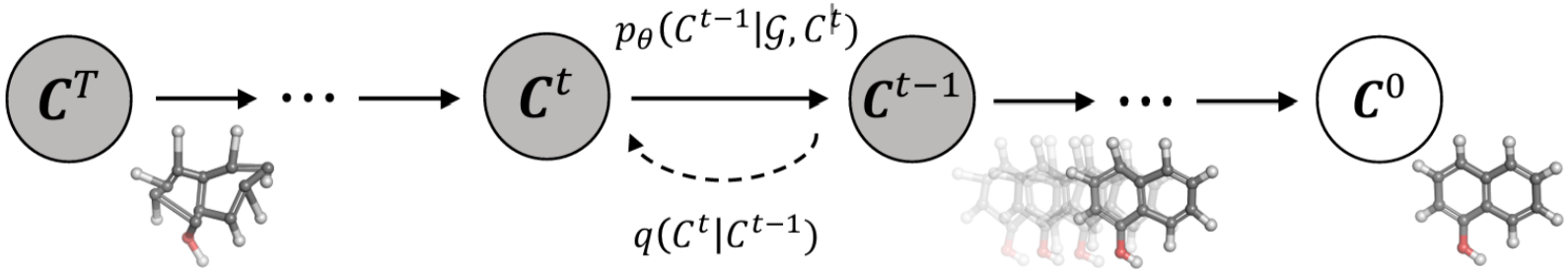
— **Diffusion model** adds noise to atomic positions (forward process)

— **Denoises random noise into realistic 3D positions** (reverse process)

— Mirrors stochastic dynamics

— Generates **diverse, low-energy conformations**

— Matches or exceeds classical methods



https://openreview.net/forum?id=PzcvxEMzvQC
https://arxiv.org/abs/2203.02923

Conceptualisation: Building a generative model for a chemistry / biology task

Thought exercise: Designing a generative model for proteins or a drug-like candidate.

In small groups discuss:

- Inputs?
- Representation?
- Constraints? (I.e. how to ensure plausibility)
- Which model type would you choose for the task?
- Evaluation metrics?